

Ingeniería del Software

Tema 5. Codificación

TABLA DE CONTENIDOS

01

Codificación

02

Código Limpio

03

Revisiones de código

04

Herramientas de ayuda a la codificación

TABLA DE CONTENIDOS

01

Codificación

02

Código Limpio

03

Revisiones de código

04

Herramientas de ayuda a la codificación

Codificación

¿Qué es la Codificación?

La actividad de Codificación, también llamada Desarrollo o Programación, transforma el diseño en código que puede ejecutarse y de esta forma ofrecer una determinada funcionalidad al usuario. Durante la codificación el diseño de la solución es concretado.

Por ejemplo, durante la actividad de diseño emerge la clase “Alumno”. El diseño concluirá propiedades y métodos de “Alumno”. El método es definido indicando su firma (parámetros de entrada y salida), pero no se codifica su contenido. Es en la actividad de codificación cuando se concretará el código del método.

Es importante subrayar que, tal como se indica en el *Tema 4 Diseño Técnico*, durante la propia actividad de codificación se puede realizar también las actividades de diseño y pruebas. El desarrollo guiado por pruebas (TDD, BDD, ATDD) hace que las actividades de diseño, codificación y testing se solapen.

¿Qué es la Codificación?

Existen diferentes paradigmas de programación. Un paradigma de programación es un modelo básico de codificación de programas. Los tres paradigmas más relevantes son:

- Programación imperativa o procedimental. Un programa se describe en términos de instrucciones, condiciones e iteraciones que modifican el estado de un programa al permitir la mutación de variables, todo esto con el objetivo de llegar a un resultado. El código está repleto de estructuras cíclicas de control tales como *while*, *do..while*, *for* para iterar sobre un determinado bloque de códigos dada una condición o expresión.
- **Programación orientada a objetos.** Un programa se describe en términos de objetos que interaccionan entre sí. Cada objeto posee su propio estado y comportamiento.
- Programación declarativa o funcional. Un programa se describe en términos de proposiciones y afirmaciones que son declaradas para describir el problema, sin especificar los pasos para resolverlo; en este tipo.

TABLA DE CONTENIDOS

01

Codificación

02

Código Limpio

03

Revisiones de código

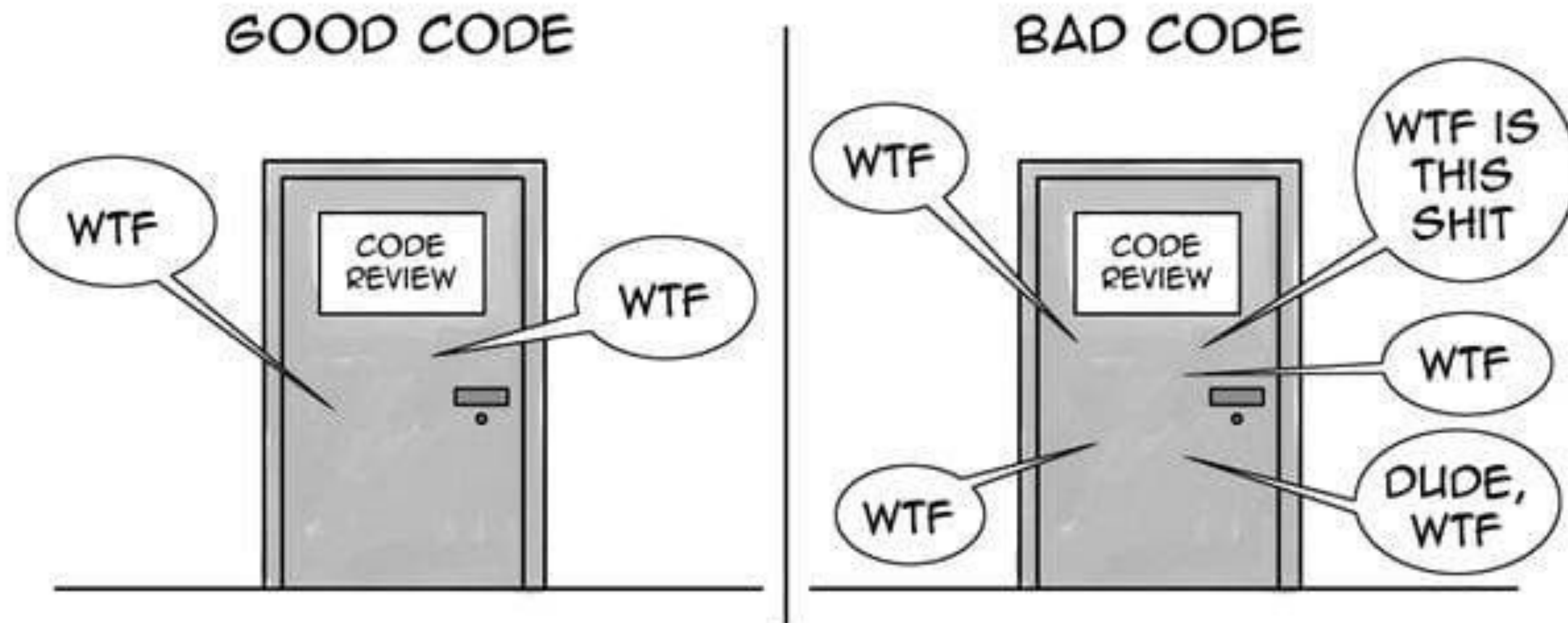
04

Herramientas de ayuda a la codificación

Código Limpio

¡No todo el código es igual!

Es posible implementar una misma funcionalidad escribiendo muy diferentes códigos. Si bien todos los códigos obtienen el mismo resultado, no todos los códigos son fácilmente mantenibles y extensibles (y por tanto testables).



THE ONLY VALID MEASUREMENT OF CODE QUALITY: WTFs/MINUTE

```
public class Alumno{
    public String Nombre;
    public int Nota;
    public int Curso;
    public Boolean SeHaLicenciado(){
        Boolean retorno = false;
        if(!Nombre.equals("")){
            if(Curso>=5){
                if(Nota>=5){
                    retorno = true;
                }
            }
            else{
                retorno = false;
            }
        }
        else{
            retorno = false;
        }
    }
    return retorno;
}
```

```
public class Alumno{
    public String Nombre;
    public int Nota;
    public int Curso;
    public Boolean SeHaLicenciado(){
        return !Nombre.equals("")
            && Nota>=5
            && Curso>=5
            ;
    }
}
```



```

(tail =>
  (is_nil =>
    (isEmpty =>
      ($0 =>
        ($1 =>
          ($2 =>
            ($3 =>
              (is_$0 =>
                ($$ =>
                  (is_$$ =>
                    (succ =>
                      (add =>
                        (pair_succ =>
                          (prev =>
                            (sub =>
                              (len =>
                                (sum =>
                                  (rcon =>
                                    (rev =>
                                      (reverse =>
                                        (elems =>
                                          (list =>
                                            (map =>
                                              map(list(elems($1)($2)($3)))(elem => sub(elem)($1))
                                              ) (y(map => l => f => when(isEmpty(l))(_ => nil)(_ => con
                                              ) (es => reverse(es($$)))
                                              ) (rcon(nil))
                                              ) (l => rev(nil)(l))
                                              ) (y(rev => r => l => when(isEmpty(l))(_ => r)(_ => rev(con(head(l))(r))
                                              ) (y(rcon => t => h => when(is_$(h))(_ => t)(_ => rcon(con(h)(t))))
                                              ) (y(sum => l => when(isEmpty(l))(_ => $0)(_ => add(head(l))(sum(tail(l)))))
                                              ) (y(len => l => when(isEmpty(l))(_ => $0)(_ => add($1)(len(tail(l)))))
                                              ) (m => n => n(prev)(m))
                                              ) (n => left(n(pair_succ)(pair(no_use)($0)))
                                              ) (p => pair(right(p))(succ(right(p)))
                                              ) (m => n => n(succ)(m))
                                              ) (n => f => x => f(n(f)(x)))
                                              ) (n => n(_ => no)(no))
                                              ) (_ => _ => yes)
                                              ) (n => n(_ => no)(yes))
                                              ) (f => x => f(f(f(x))))
                                              ) (f => x => f(f(x)))
                                              ) (f => x => f(x))
                                              ) (_ => x => x)
                                            ) (is_nil)
                                          ) (l => l(_ => _ => no))
                                        ) (right)

```



¡No todo el código es igual!

“[...] de hecho, la relación entre el tiempo dedicado a leer y escribir es más de 10 a 1. Estamos constantemente leyendo código antiguo como parte del esfuerzo por escribir código nuevo. [Por tanto,] facilitar la lectura facilita la escritura”.

Robert C. Martin



¿Qué es el código espagueti?

El **código espagueti** es un término peyorativo para los programas de computación que tienen una estructura de control de flujo compleja e incomprensible. Su nombre deriva del hecho que este tipo de código parece asemejarse a un plato de espaguetis, es decir, un montón de hilos intrincados y anudados

```
template <typename T>
void goto_sort( T array[], size_t n ) {
    size_t i{1}

    first: T current{array[i]};
    size_t j{i};

    second: if ( array[j - 1] <= current ) {
        goto third;
    }

    array[j] = array[j - 1];

    if ( --j ) {
        goto second;
    }

    third: array[j] = current;

    if ( ++i != n ) {
        goto first;
    }
}
```

¿Qué son los “code smells”?

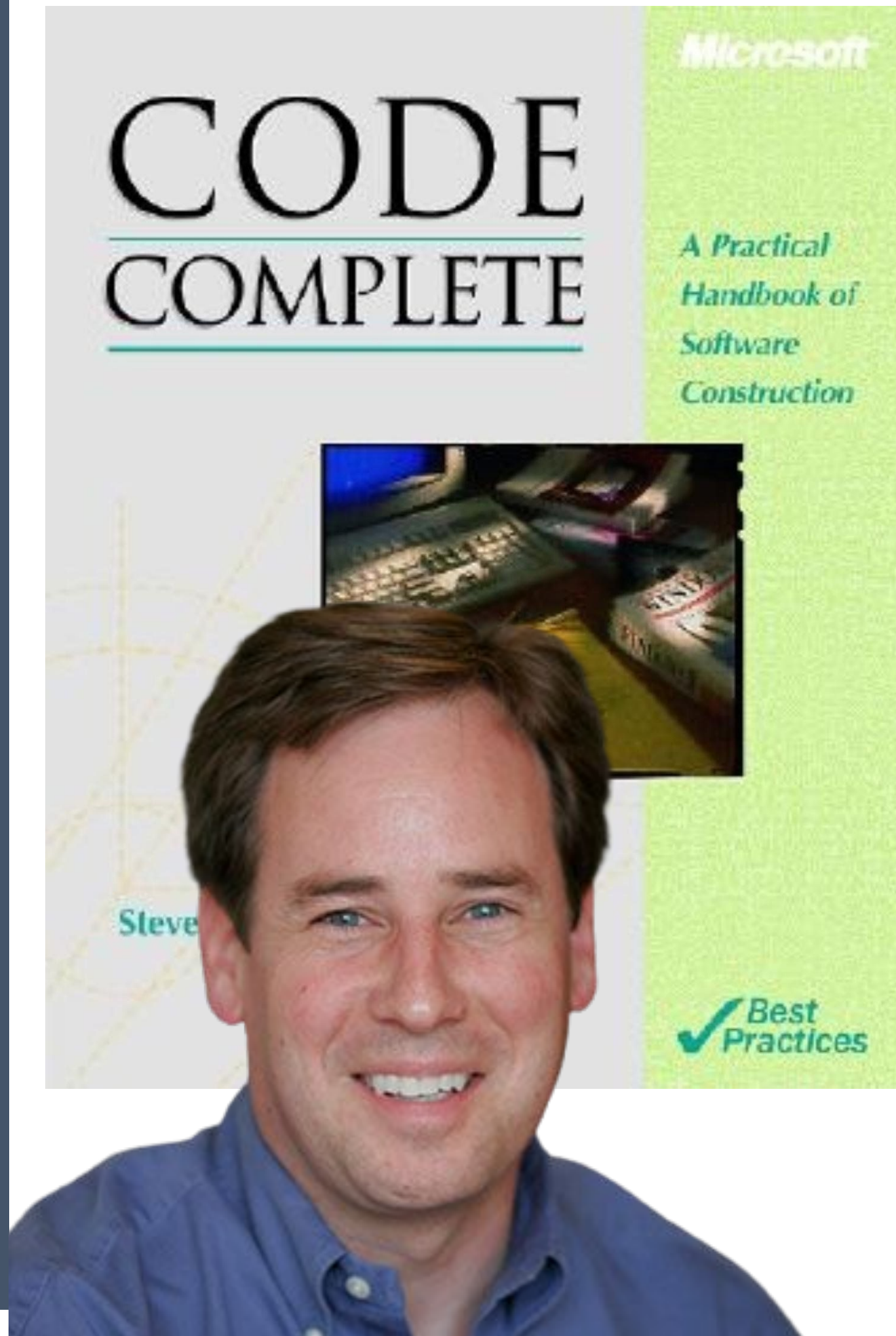
- ❑ **Métodos o clases muy grandes.** Representan algo que ha crecido en proporciones tan grandes que resulta complejo tanto de entender como de manejar.
- ❑ **Obsesión por los tipos primitivos.** Los únicos tipos utilizados son los que ofrece el lenguaje de forma nativa (Integer, String, Boolean).
- ❑ **Métodos con muchos parámetros de entrada.** Al no escribir nuevas clases, son necesarios demasiados parámetros para que el método tenga lo que precisa para actuar.
- ❑ **No hacer uso ni de la herencia ni del polimorfismo.** Las clases son usadas para estructurar datos, no para modelar el negocio.
- ❑ **Abuso de variables globales.** Diferentes componentes del software hacen uso de las mismas variables globales para persistir el estado del software o comunicar eventos.

“El camino al infierno de la programación está pavimentado con variables globales.”

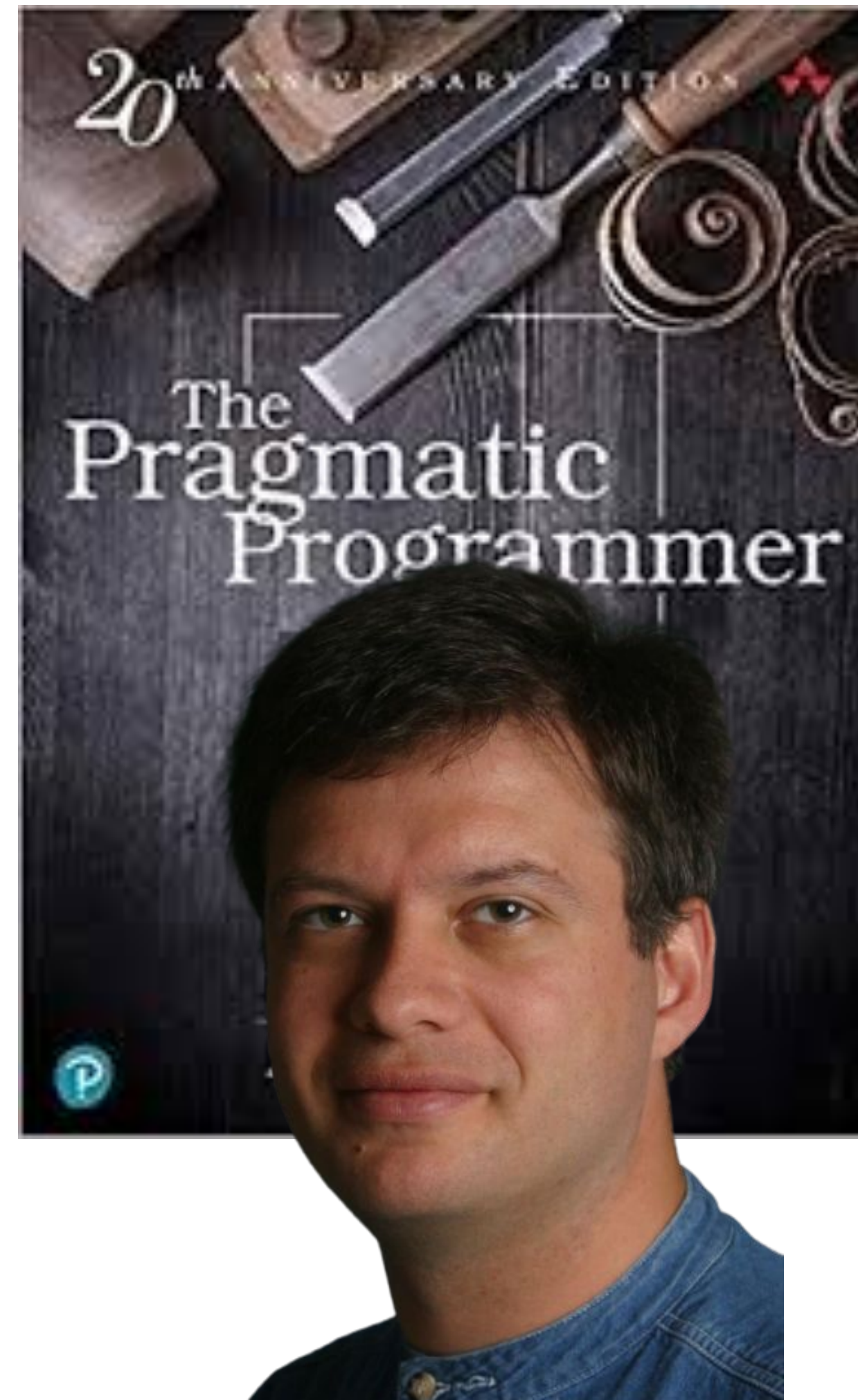
Steve McConnell, Code Complete

¿Qué es el “Código Limpio”?

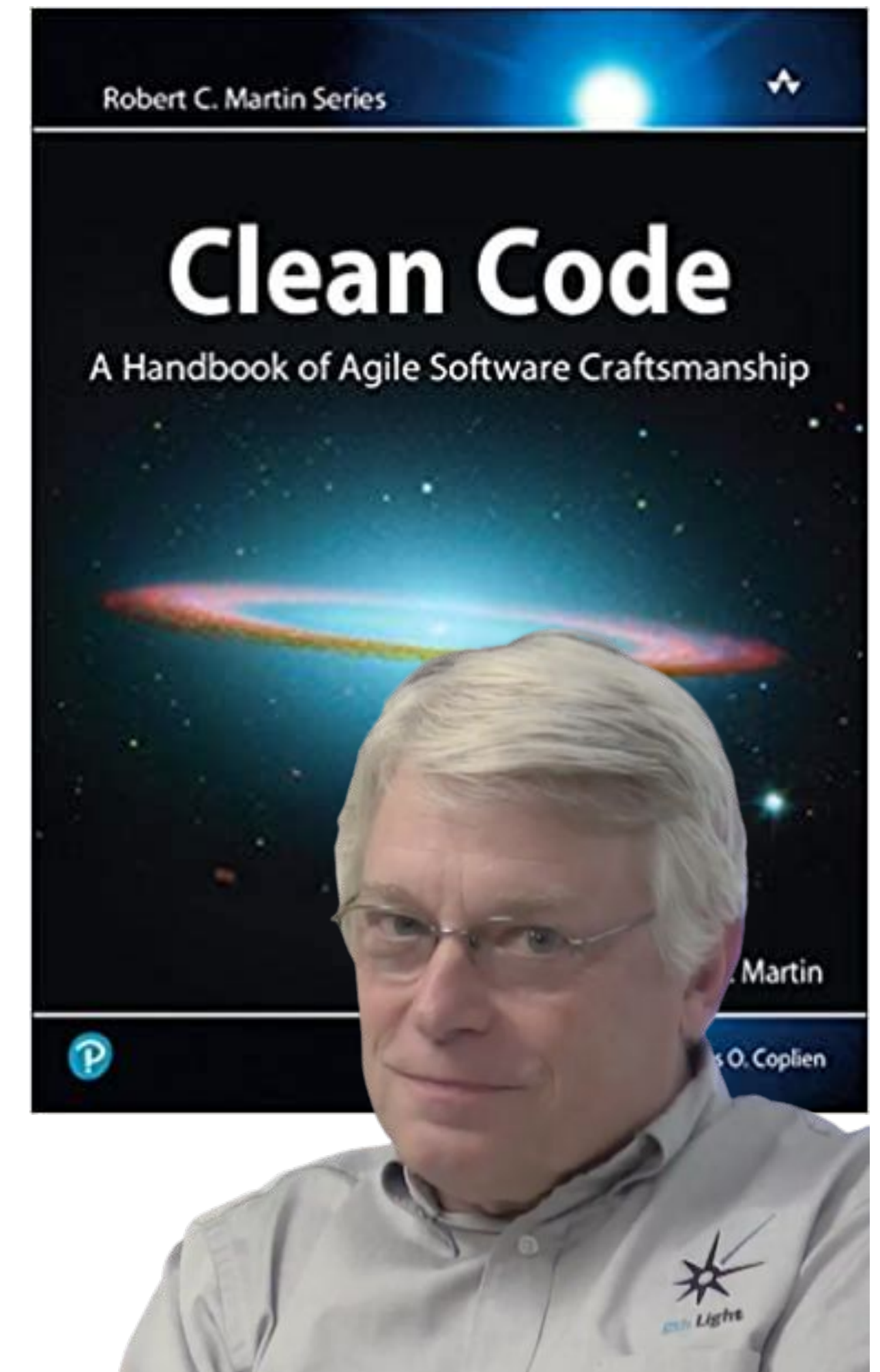
Steve McConnell
Code Complete 1993



Andrew Hunt
The Pragmatic Programmer 1999



Robert C. Martin
Clean Code 2008



¿Qué es el “Código Limpio”?

- ❑ **El código limpio se lee como prosa bien escrita.** El código debe ser agradable: los nombres de variables deben ser reveladores, el flujo de control debe ser sencillo. El código limpio cuenta una historia.
- ❑ **El código limpio es sencillo y conciso.** Debe evitar flujos complejos, abstracciones innecesarias, o división de funciones excesiva. Es mejor el código simple que el complejo.
- ❑ **El código limpio es honesto.** El código debe mostrar claramente sus intenciones, sin ambigüedades, sin sorpresas, sin efectos colaterales. Por ejemplo, Si se invoca la función "fillComboBox", no se espera escribir un fichero.
- ❑ **El código limpio expresa lenguaje del dominio del negocio.** Habla de los problemas y no de las soluciones. Por ejemplo, en vez de *print(inputRecord)*, es preferible *sendPromotionLetterTo(employee)*. *print(inputRecord)* puede ser escrito en cualquier ámbito, tanto para enviar una carta de promoción a un empleado como para escribir un registro traza en un fichero de log. El código *print(inputRecord)* no habla del negocio.
- ❑ **El código limpio es fácilmente testable.** El programador debe poder ejecutar segmentos de código sin necesidad de “montar” la aplicación completa.

¿Qué es el “Código Limpio”?

- ❑ **Naming.** El nombre de variables, objetos, tipos y clases debe ser suficientemente ilustrador. Deben seguir las convenciones de codificación del lenguaje. Las convenciones son un conjunto de normas para un lenguaje de programación específico que recomiendan estilos, buenas prácticas y métodos para mantener el aspecto del código fuente. Estas convenciones incluyen la organización de archivos, la indentación, los comentarios, las declaraciones, los espacio en blanco, las llaves de apertura y cerrado... Ejemplos de convenciones: CamelCase, snake_case, notación húngara (bStopped), etc.



Leon “Jaggleson” Bambrick
@secretGeek



There are 2 hard problems in computer science: cache invalidation, naming things, and off-by-1 errors.

3:20 p. m. · 1 ene. 2010 · Twitter Web Client

1.420 Retweets 165 Tweets citados 1.716 Me gusta

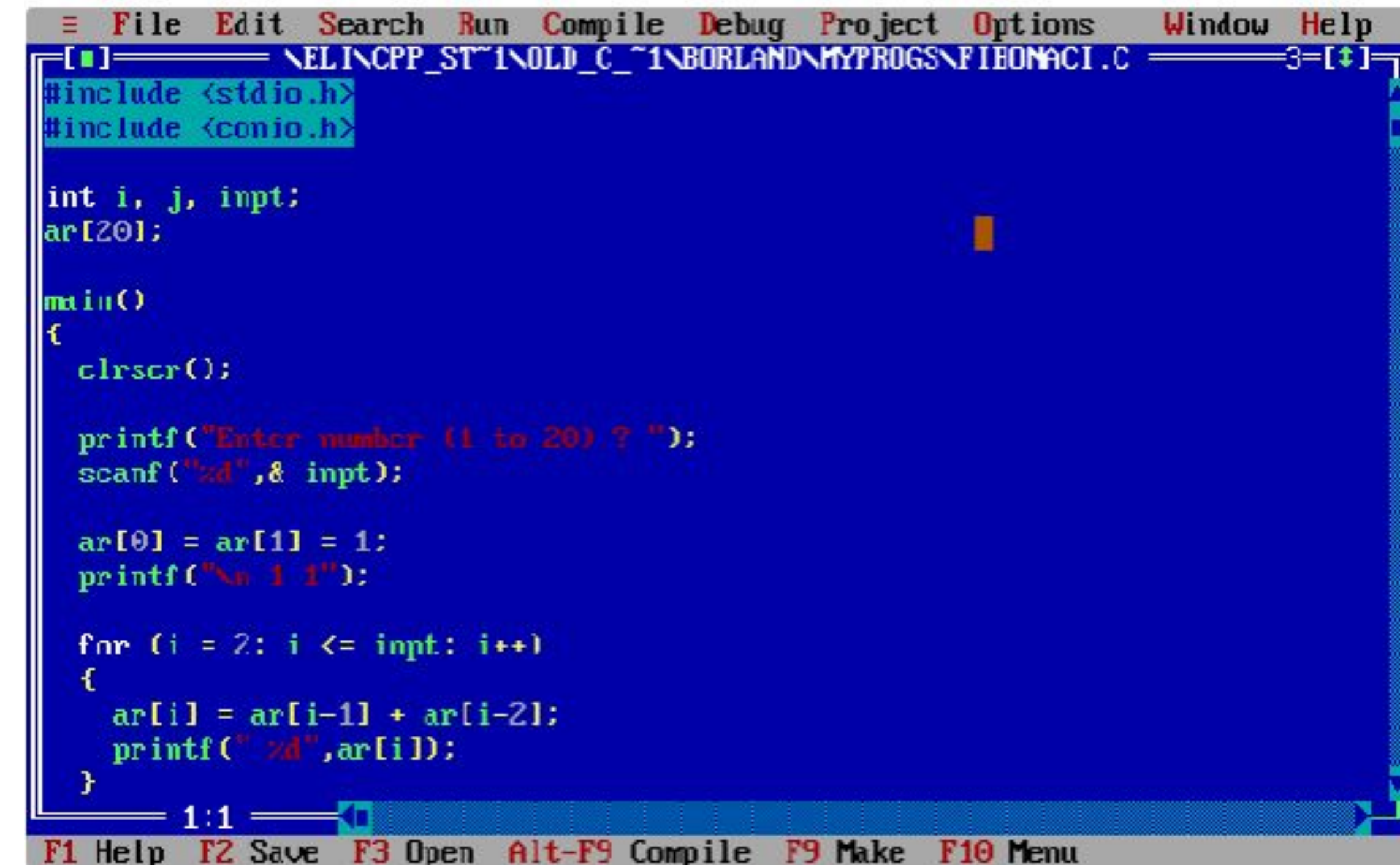
Naming things is hard

¿Qué es el “Código Limpio”?

- ❑ **Funciones pequeñas.** Deben ser reducidas apenas unas decenas de líneas y con nombres descriptivos (sin importar su longitud). Deben evitar el anidamiento excesivo (funciones que invocan otras funciones que a su vez invocan otras funciones). No deberían tener más de 2 o 3 parámetros.
- ❑ **Comentarios.** Los comentarios solo están justificados cuando no somos capaces de expresarnos con el código. Responden al “por qué”.
- ❑ **Formato.** Debe establecerse un orden dentro de la clase, por ejemplo constantes estáticas, variables estáticas, variables de instancia y funciones.
- ❑ **Finish what you start.** Este principio recomienda que el método que solicita un recurso también debe ser quien lo libere.
- ❑ **Crash Early.** Cuando algo inesperado ocurre dentro del sistema se debe detener la ejecución y evitar seguir.
- ❑ **Efecto Stroop.** Se produce cuando los nombres empleados son engañosos. Por ejemplo, que un método denominado `escribeFichero()` solicite datos al usuario para calcular una tarifa de un producto.

¿Qué es el “Código Limpio”?

La *intelligence* de los nuevos IDEs, ha hecho caducar la antigua buena práctica “una función solo debe tener un punto de salida”. Hoy, los IDEs ayudan al desarrollador a detectar bifurcaciones de código que acaban sin respuesta. De esta forma, lo que antes se consideraba una buena práctica hoy es considerado un antipatrón. La buena práctica actual consiste en escribir “cláusulas de guardia”, ofreciendo múltiples puntos de salida, facilitando la comprensión del código.



```
File Edit Search Run Compile Debug Project Options Window Help
\\ELINCPP_ST~1\OLD_C_~1\BORLAND\MYPROGS\FIBONACI.C 3=1
#include <stdio.h>
#include <conio.h>

int i, j, inpt;
ar[20];

main()
{
    clrscr();

    printf("Enter number (1 to 20) ? ");
    scanf("%d",& inpt);

    ar[0] = ar[1] = 1;
    printf("\n 1 1");

    for (i = 2; i <= inpt; i++)
    {
        ar[i] = ar[i-1] + ar[i-2];
        printf(" %d",ar[i]);
    }
}
```

F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

De forma similar, en los 80s y 90s se recomendaba escribir abundantes comentarios. La aparición de técnicas de desarrollo iterativo guiado por pruebas hacen prescindibles estos comentarios. Actualmente la abundancia de comentarios es considerada una mala práctica que delata la existencia de código espagueti.



TABLA DE CONTENIDOS

01

Codificación

02

Código Limpio

03

Revisiones de código

04

Herramientas de ayuda a la codificación

Revisiones de Código

¿Qué son las “revisiones de código”?

Una revisión de código (Core Review) es un proceso en el cual es código es revisado por alguien diferente al propio autor. Gracias a las revisiones de código:

- El código debe obligatoriamente ser leído y comprendido por, al menos, otro desarrollador.
- El autor del código recibe “feedback” durante la codificación, lo cual le puede ayudar a mejorar sus competencias técnicas.
- El conocimiento es compartido en el equipo.

¿Qué son las “revisiones de código”?

La revisión del código suele realizarse antes de contribuir el código a la base de código. Típicamente una revisión de código sigue los siguientes pasos (*Software Engineering at Google, 2020*):

1. Un desarrollador escribe un cambio en la base de código en su propio espacio de trabajo “workspace”. El autor etiqueta el cambio realizado añadiendo una pequeña descripción.
2. El desarrollador puede realizar una “auto-revisión” de código haciendo uso de analizadores de código. Cuando el desarrollador está satisfecho con el cambio, solicita a uno o varios revisores la revisión del código.
3. Los revisores, haciendo uso de herramientas de revisión de código, envían comentarios y dudas sobre el código al autor.
4. El desarrollador realiza las modificaciones solicitadas y/o resuelve las dudas de los revisores. Los pasos 3 y 4 pueden repetirse varias veces.
5. Los revisores aceptan el cambio. Cada revisor debe etiquetar el código como “suficientemente bueno” o “*looks good to me*” (LGTM).
6. El desarrollador contribuye el código a la base.

¿Qué son las “revisiones de código”?

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#).

base repository: **99xt-incubator/articles-of-the-week**

base: **master**

head repository: **ThanoshanMV/articles-of-the-week**

✓ **Able to merge.** These branches can be automatically merged.

Adding an article to week 02 of articles of the week

Write

Preview

A **B** *i* “ ” < > ↺ ⋮ ⋮ ✓ @ 🚩 ↶

Leave a comment

Attach files by dragging & dropping, selecting or pasting them.

☒ **Allow edits from maintainers.** [Learn more](#)

Create pull request

▼

🔗 1 commit

📄 1 file changed

	@@ -234,9 +234,10 @@
// Opacity	// Opacity
.opacity(@opacity: 100) {	.opacity(@opacity: 100) {
filter: e(%("alpha(opacity=%d)", @opacity));	filter: e(%("alpha(opacity=%d)", @opacity));
- -khtml-opacity: @op / 100;	+ -khtml-opacity: @opacity / 100;
- -moz-opacity: @op / 100;	+ -moz-opacity: @opacity / 100;
- opacity: @op / 100;	+ opacity: @opacity / 100;
	+ -ms-filter: e(%("progid:DXImageTransform.Microsoft.Alpha(Opacity=%d)", @opacity));
}	}
// CSS3 Flexible Box Module	// CSS3 Flexible Box Module
	@@ -289,4 +290,4 @@
-webkit-box-pack: @pack;	-webkit-box-pack: @pack;
box-pack: @pack;	box-pack: @pack;
}	}
-}	+}

TABLA DE CONTENIDOS

01

Codificación

02

Código Limpio

03

Revisiones de código

04

Herramientas de ayuda a la codificación

Herramientas de Ayuda a la Codificación

- Entorno de Desarrollo Integrado (IDE)
- Depuradores
- Control de Versiones
- Integración Continua
- Frameworks
- Analizadores de Código
- Autogeneradores de Código

Herramientas de Ayuda a la Codificación

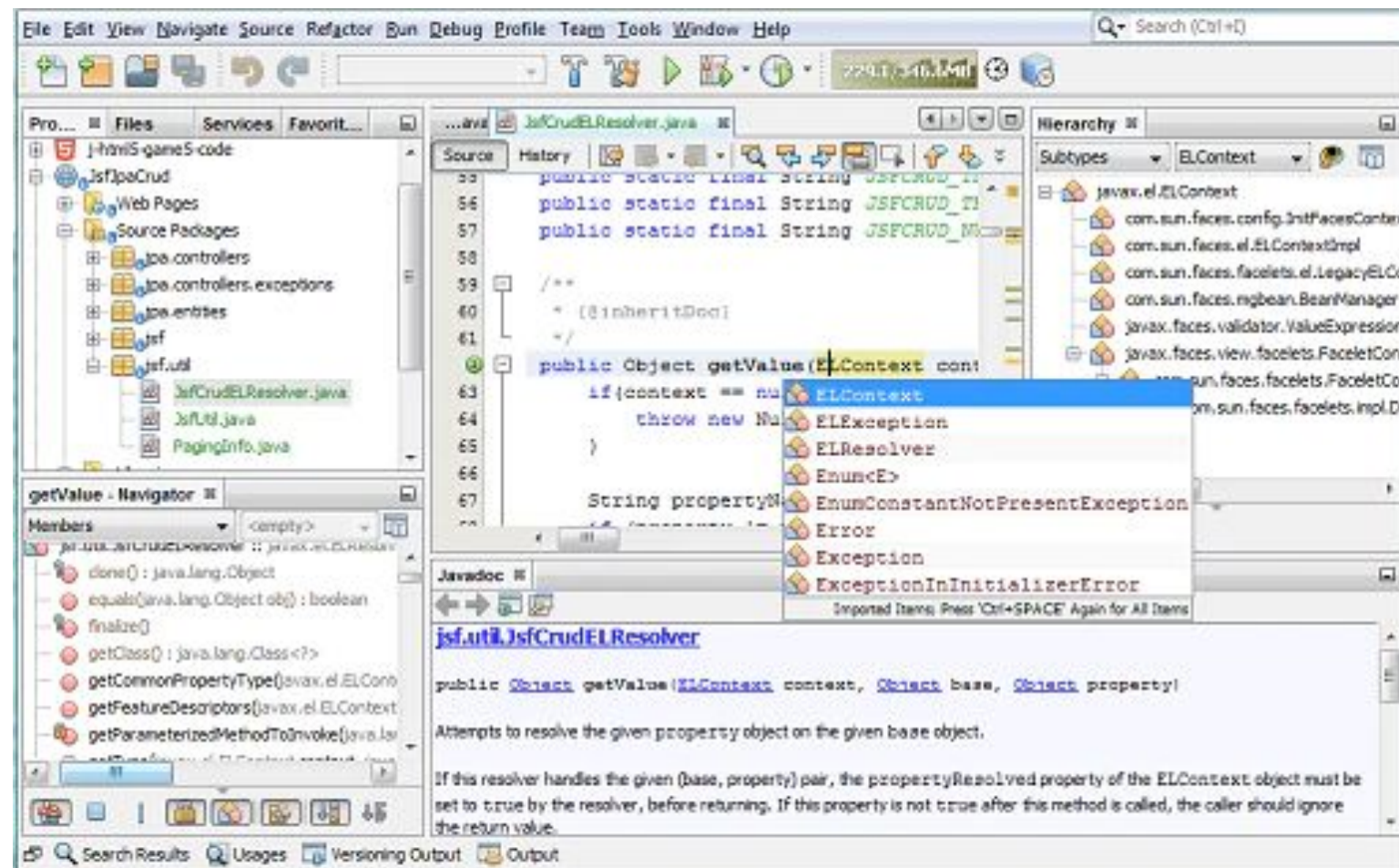
- **Entorno de Desarrollo Integrado (IDE)**
- **Depuradores**
- **Control de Versiones**
- **Integración Continua**
- **Frameworks**
- **Analizadores de Código**
- **Autogeneradores de Código**

¿Qué es el “IDE”?

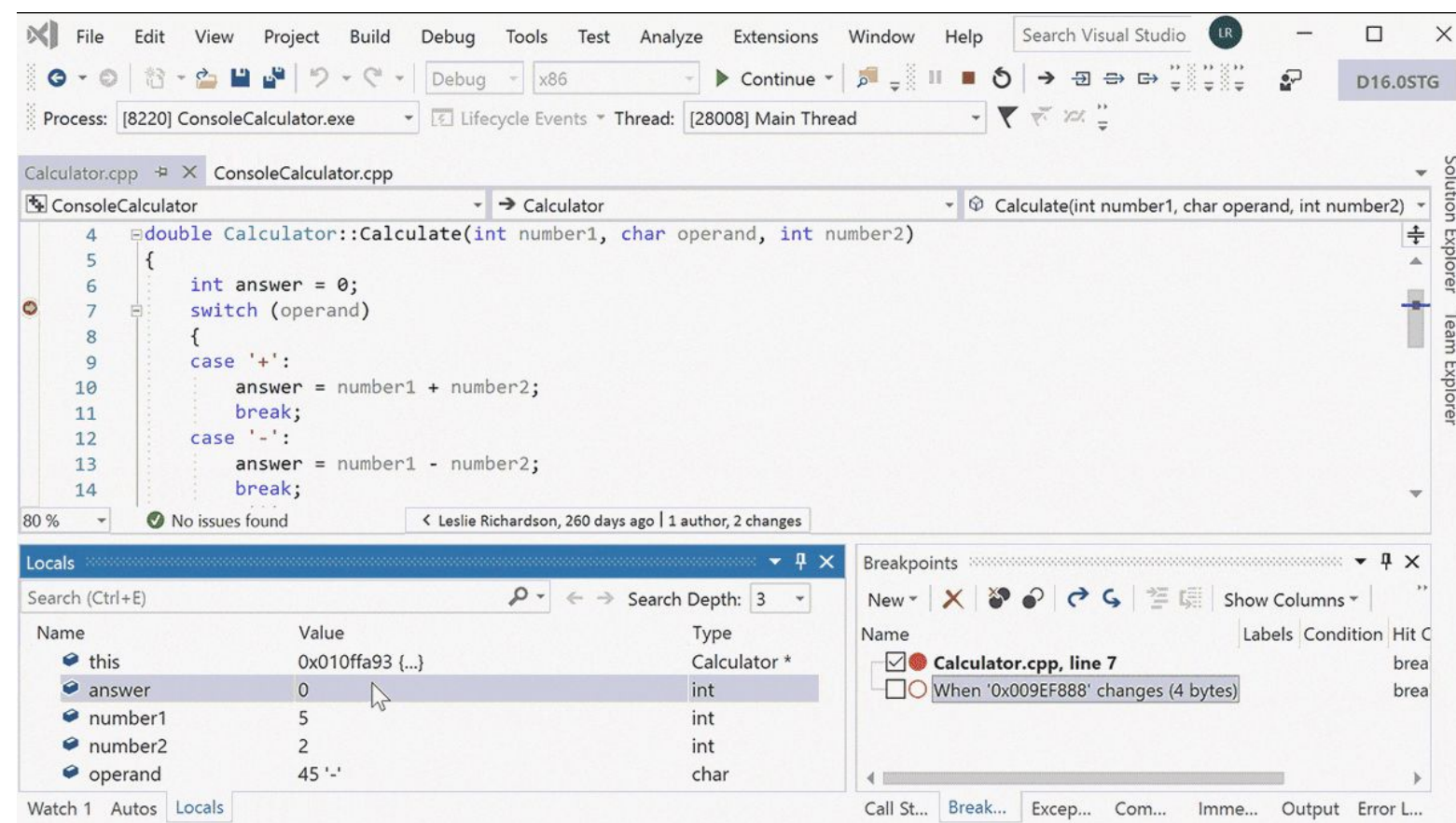
Un entorno de desarrollo integrado o entorno de desarrollo interactivo, en inglés Integrated Development Environment (IDE), es una aplicación informática que proporciona servicios integrales para facilitarle al programador el desarrollo de software.

Normalmente, un IDE consiste de un editor de código fuente, herramientas de construcción automáticas y un depurador. La mayoría de los IDE tienen auto-completado inteligente de código (IntelliSense).

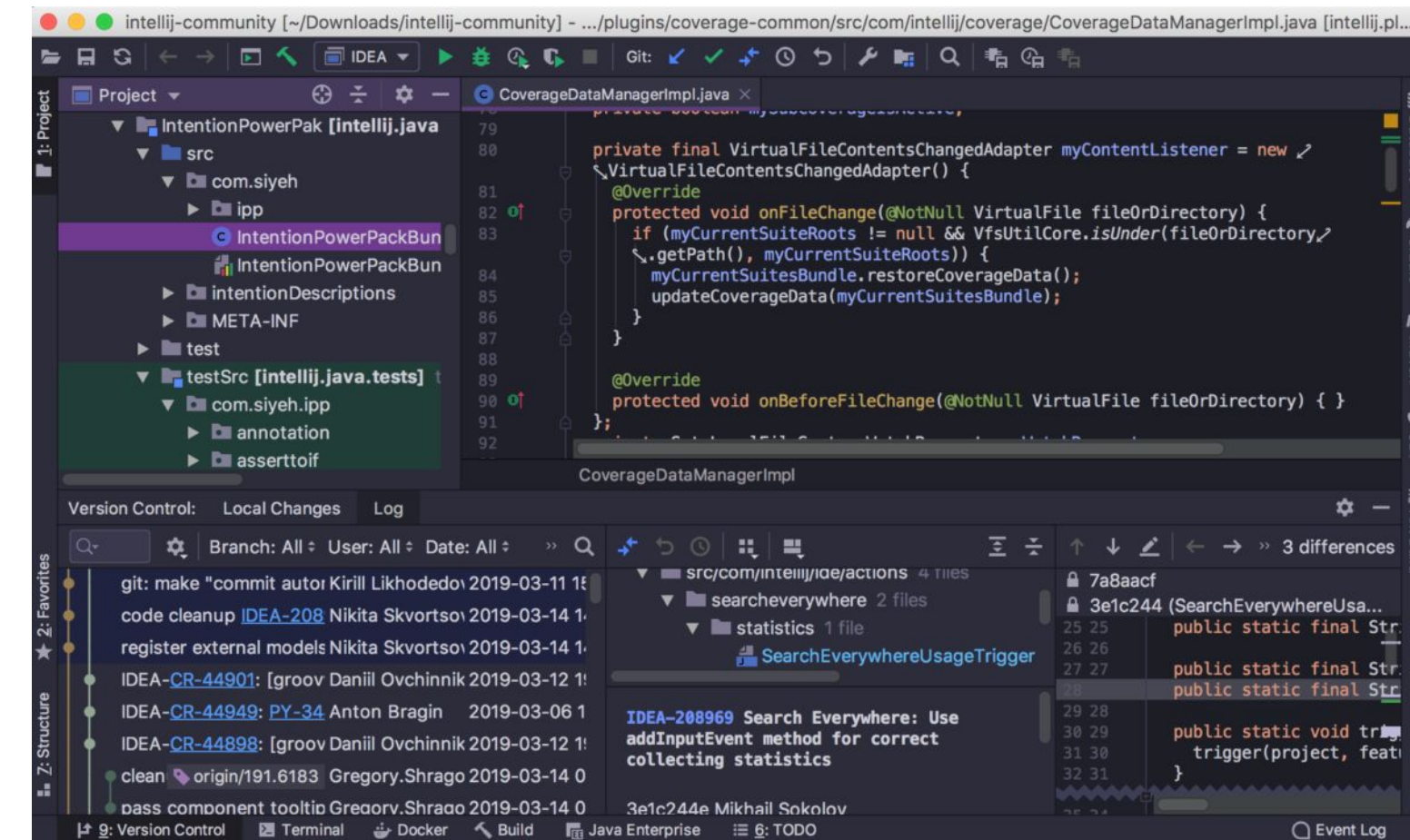
¿Qué es el “IDE”?



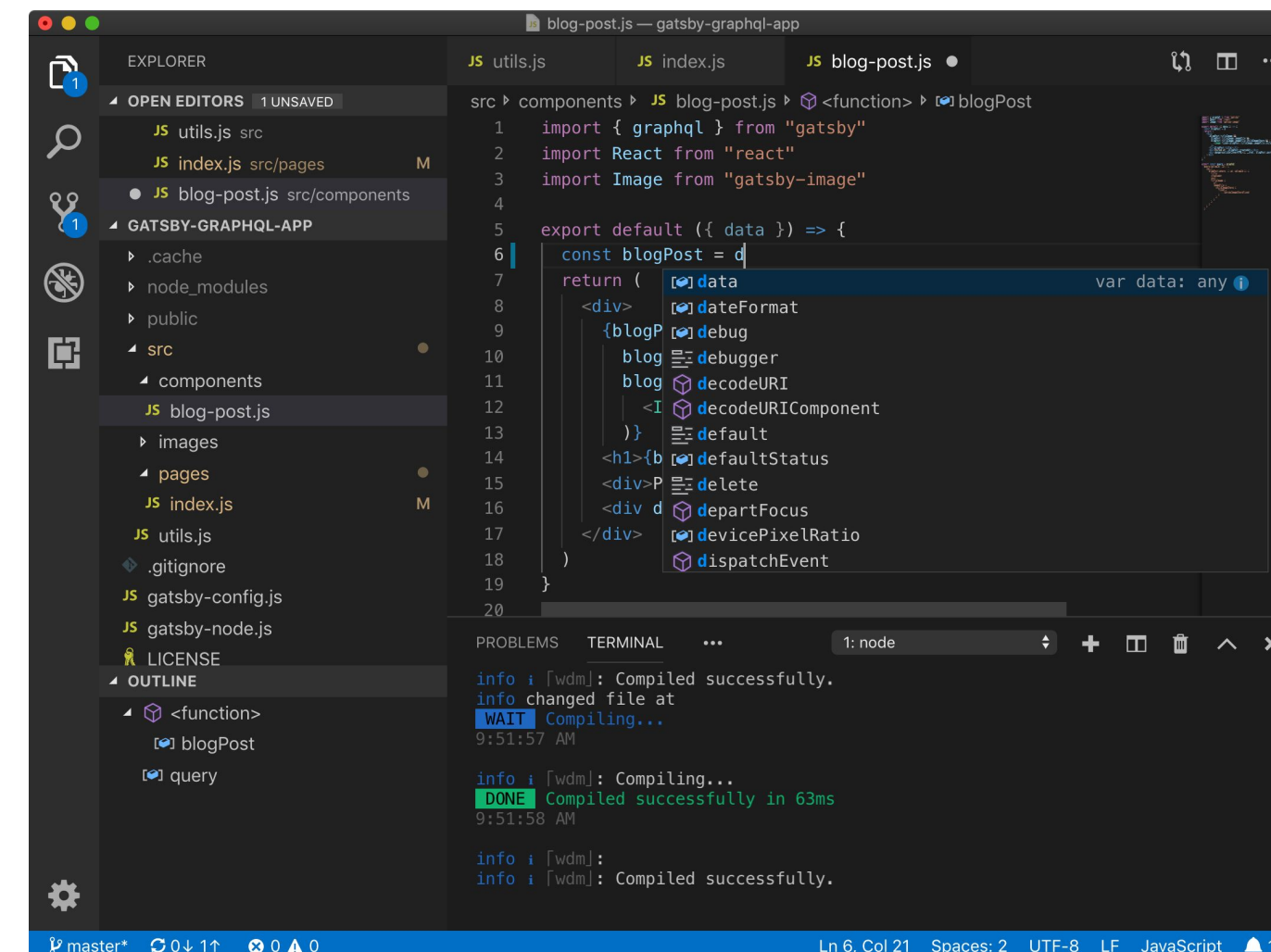
Netbeans



Visual Studio

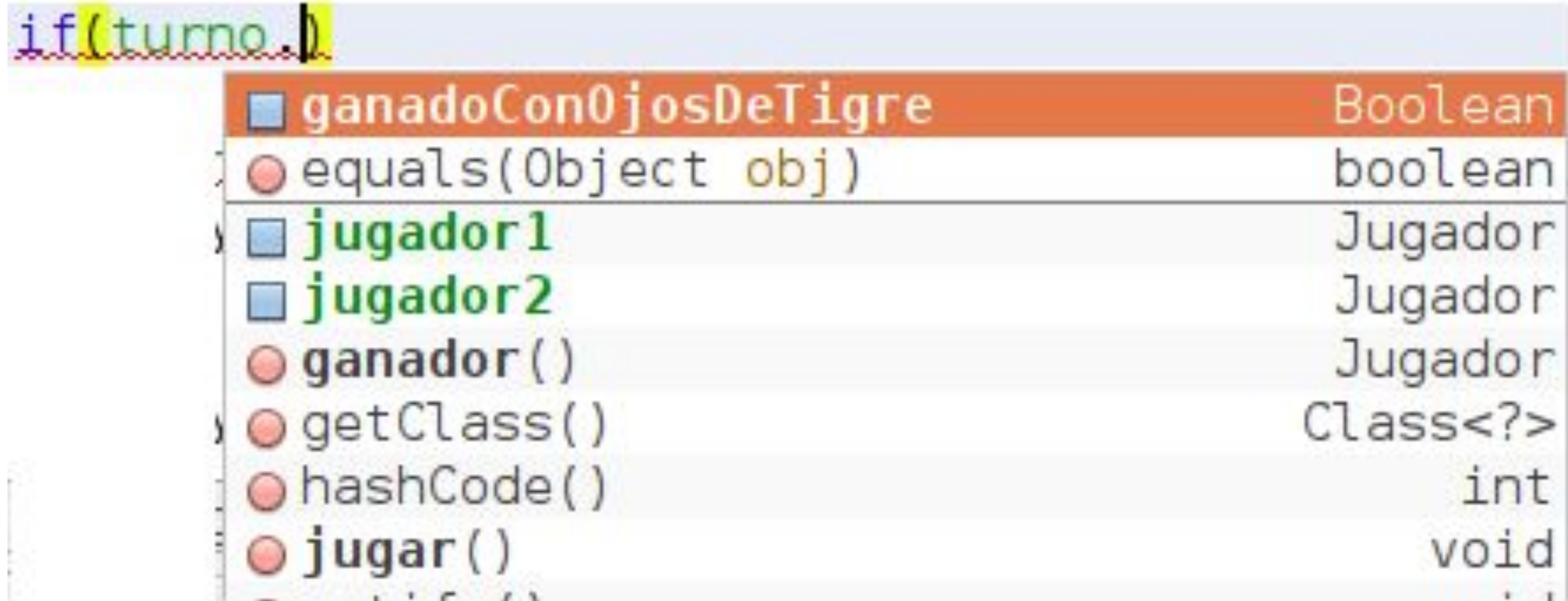


IntelliJ



VS Code

Algo más que autocompletar



AUTOCOMPLETADO CON NETBEANS

Algo más que autocompletar

```
1 #!/usr/bin/env ts-node
2
3 import { fetch } from "fetch-h2";
4
5 // Determine whether the sentiment of text is positive
6 // Use a web service
7 async function isPositive(text: string): Promise<boolean> {
8   const response = await fetch(`http://text-processing.com/api/sentiment/`, {
9     method: "POST",
10    body: `text=${text}`,
11    headers: {
12      "Content-Type": "application/x-www-form-urlencoded",
13    },
14  });
15  const json = await response.json();
16  return json.label === "pos";
17 }
```

 Copilot

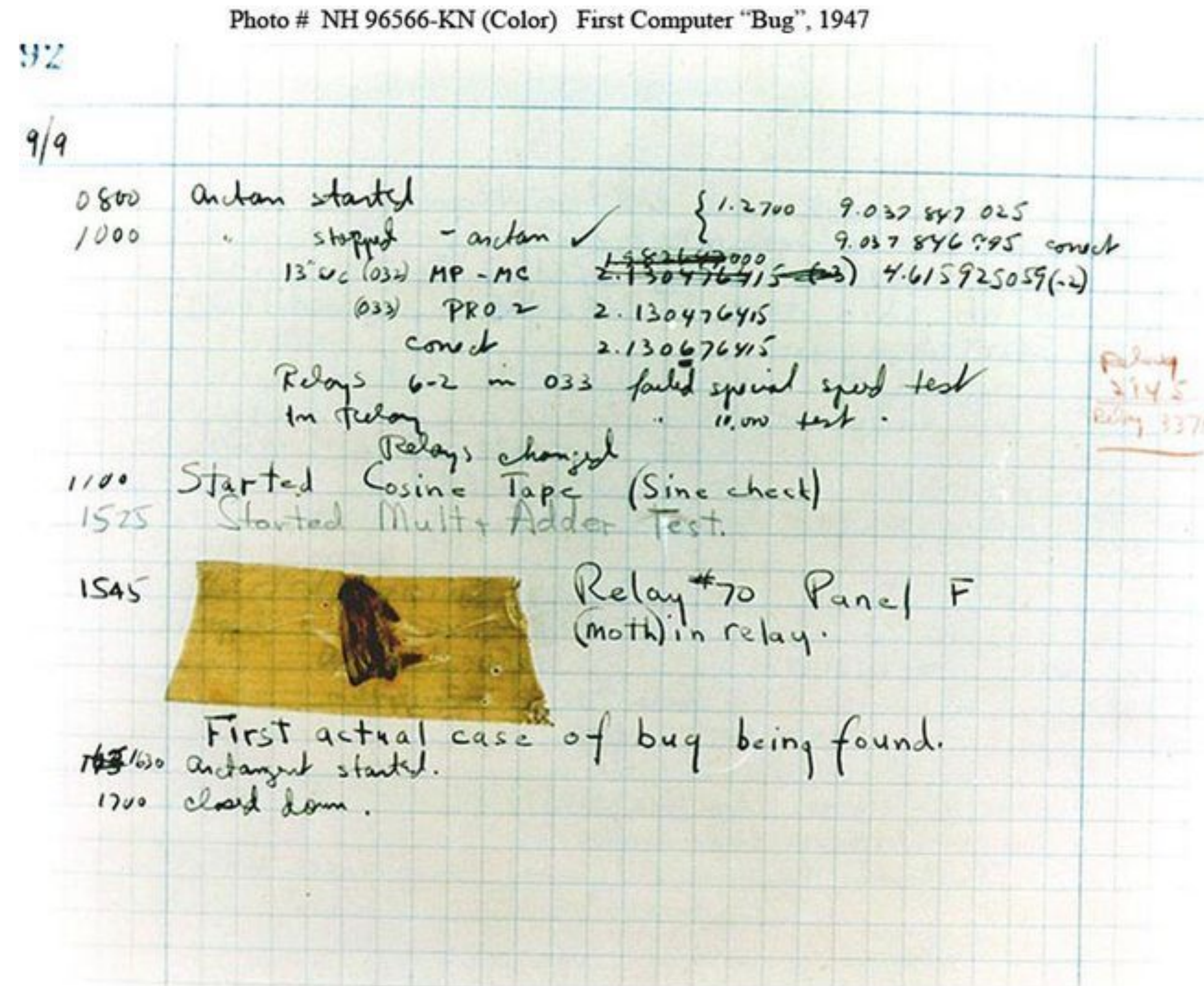
GITHUB COPILOT

Herramientas de Ayuda a la Codificación

- Entorno de Desarrollo Integrado (IDE)
- **Depuradores**
- Control de Versiones
- Integración Continua
- Frameworks
- Analizadores de Código
- Autogeneradores de Código

¿Qué es un bug?

Uno de los principales problemas al desarrollar aplicaciones son los errores de ejecución. Compilar un programa no es garantía suficiente de que funciona de la manera prevista. A los errores de ejecución en programas se les suele denominar en inglés bugs. El término “debug” se inició en 1947, haciendo referencia a quitar todos los insectos “bugs” que se colaban dentro de las válvulas de vacío de las primeras computadoras.

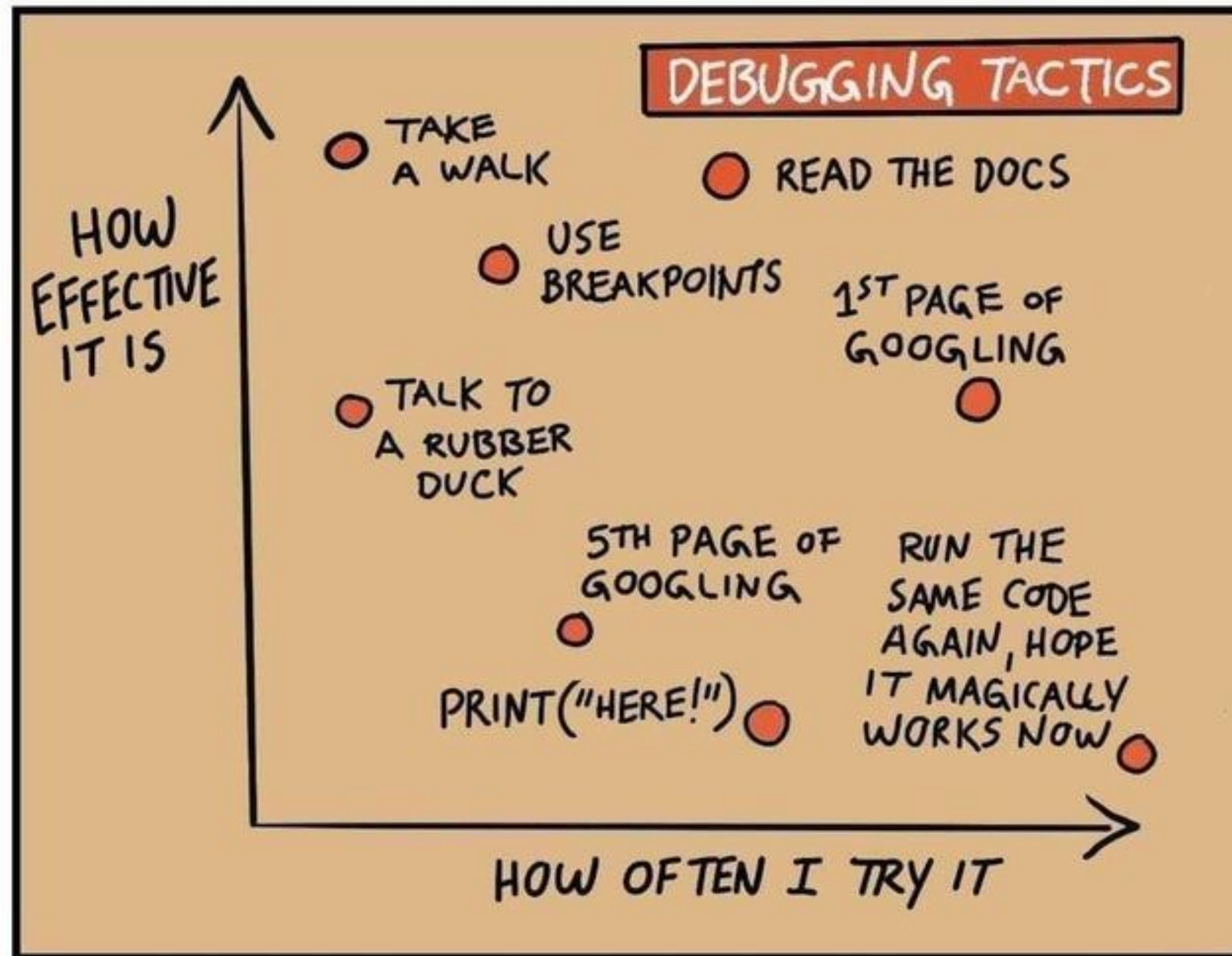


Tipos de Debuggers

Un Debugger (o depurador) es un programa cuyo objetivo es el de probar y eliminar (depurar) errores de un programa objetivo. Para ello, permite a los programadores seguir paso a paso (instrucción a instrucción) la ejecución del programa objetivo.

- ❑ Un depurador de **alto nivel** también llamado depurador simbólico es un debugger que toma el control sobre el programa para poder determinar su estado, estableciendo breakpoints (puntos de pausa/ruptura dentro de la ejecución). Este tipo de debugger es el comúnmente utilizado por los entornos de desarrollo integrados (IDE).
- ❑ Un depurador de **bajo nivel** o de lenguaje máquina se centra en el código ensamblado. Este debugger se centra en problemas como que una instrucción no sea válida para la CPU donde se está ejecutando el programa objetivo o si se intenta acceder a espacio utilizado por memoria protegida

Técnicas de Debugging (meme)



Herramientas de Ayuda a la Codificación

➤ Entorno de Desarrollo Integrado (IDE)

➤ Depuradores

➤ **Control de Versiones**

➤ Integración Continua

➤ Frameworks

➤ Analizadores de Código

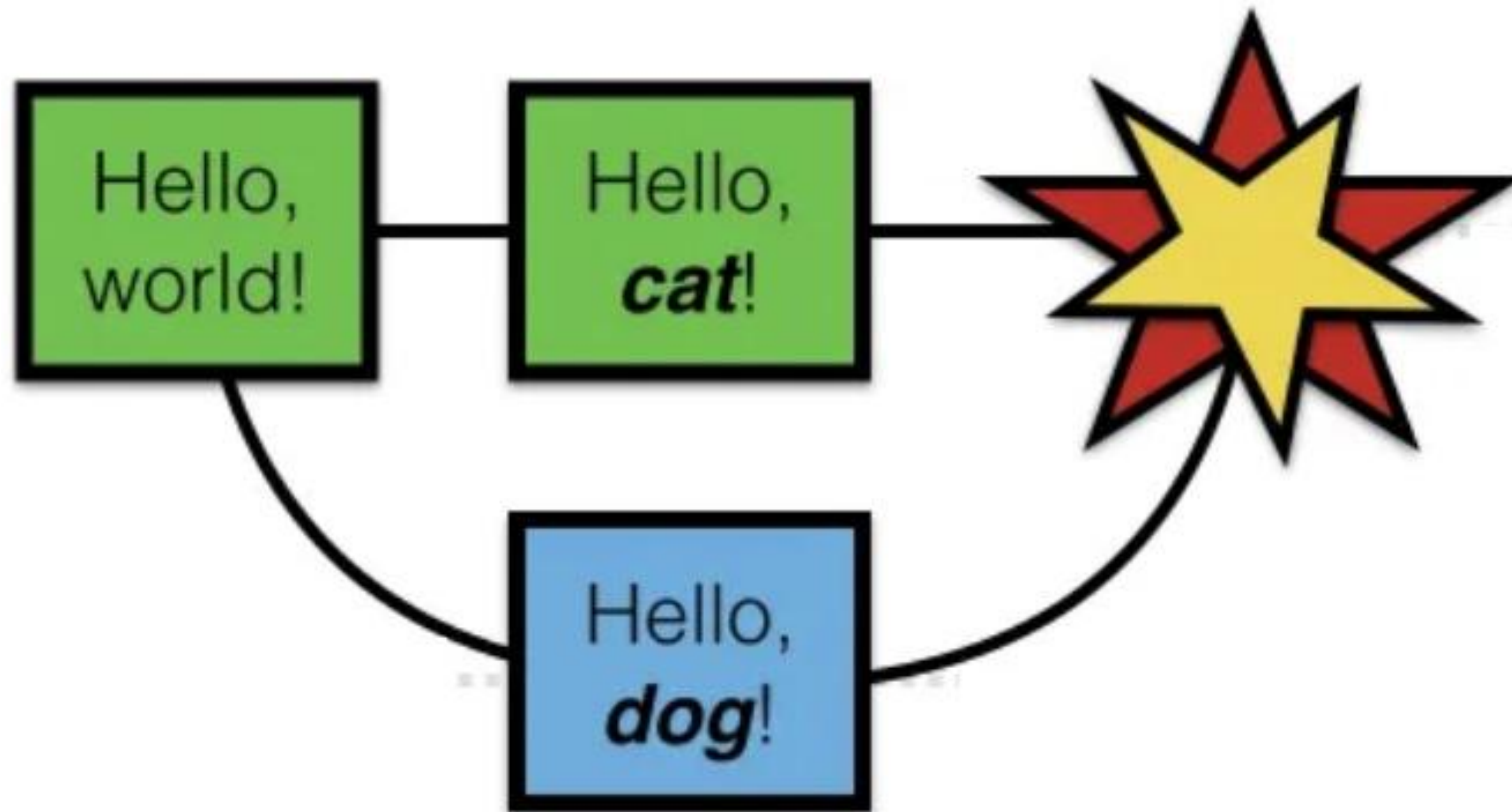
➤ Autogeneradores de Código

¿Qué es el Sistema de Control de Versiones?

Un control de versiones o VCS (del inglés Version Control System) es un sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo, de modo que puedan ser recuperadas versiones específicas más adelante.

En el desarrollo de software se utilizan los sistemas de control de versiones para gestionar los cambios de los archivos de código fuente.

¿Qué es el Sistema de Control de Versiones?



Características de Sistema de Control de Versiones

Todos los sistemas de control de versiones cuentan con un repositorio. El repositorio es el almacén de archivos capaz de guardar todo el historial de cambios.



Centralizados. Es un sistema que permite a los usuarios trabajar en un proyecto común compartido a través de un único servidor central que funciona como un punto de sincronización común. Esta configuración tiene como principal desventaja la dependencia con el servidor central. Un ejemplo es Subversion.

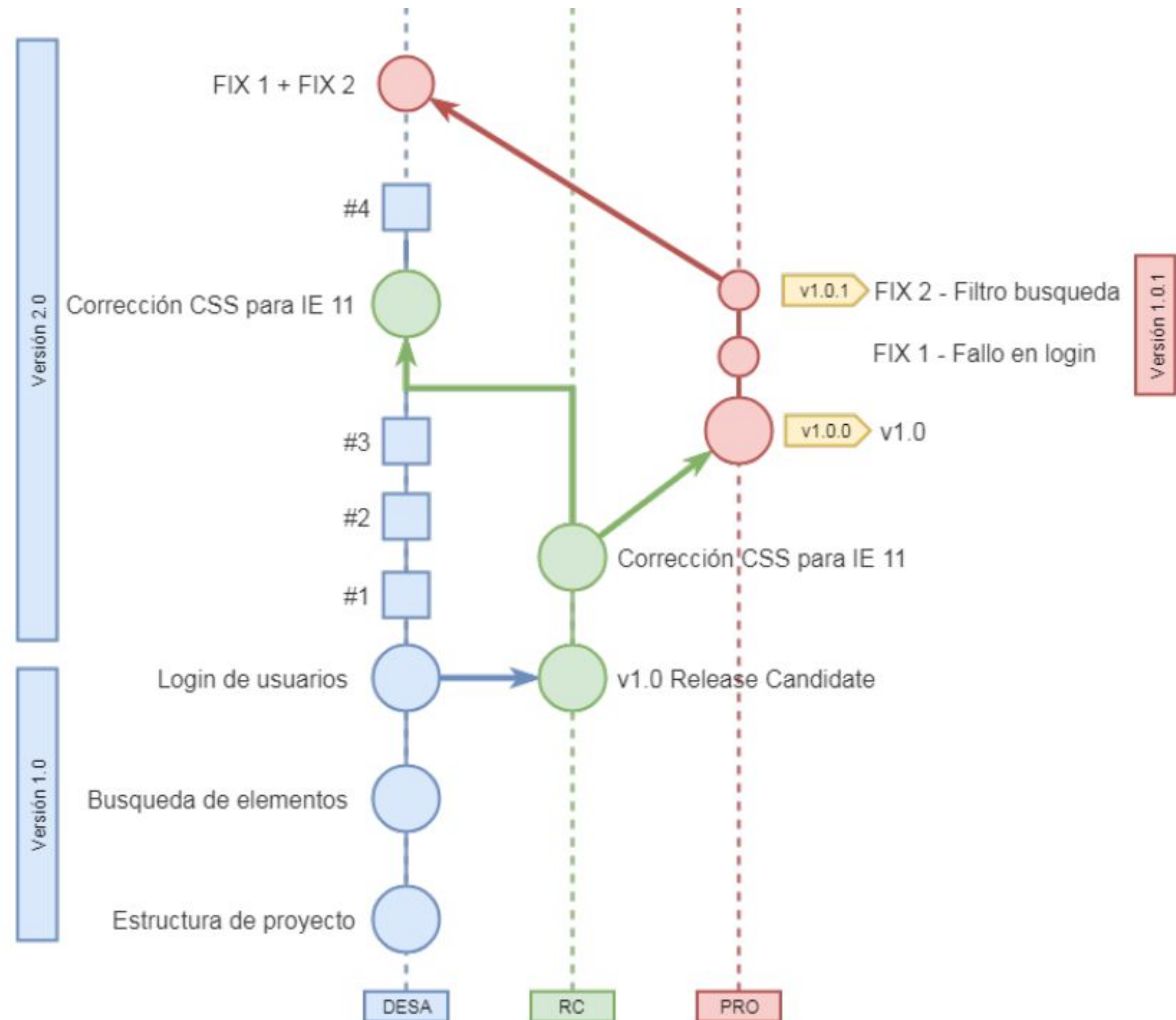


Distribuidos. Cada desarrollador clona el repositorio en su máquina local. Las contribuciones de código son realizadas en el repositorio local de forma desconectada del resto del equipo. Una vez el desarrollador ha terminado el trabajo se contribuye el repositorio a un servidor central, momento en el que se detectan los posibles conflictos. En cada máquina local existe una copia completa de todo el historial de cambios del código, incluidas ramas. Un ejemplo es GIT.

Ventajas del Sistema de Control de Versiones

- ❑ **Completo historial de cambios de todos los archivos.** Los cambios incluyen la creación y la eliminación de los archivos, así como los cambios de sus contenidos. El historial de cambios incluye el autor, la fecha y notas escritas sobre el propósito de cada cambio.
- ❑ **Gestión de ramas.** Una "rama" permite mantener múltiples versiones del proyecto independientes unas de otras. Los VCS permiten fusionar "ramas" verificando que los cambios de una rama con los de otra no entran en conflicto.
- ❑ **Trazabilidad.** Los VCS permiten conectar cada cambio que se hace en el código con herramientas de gestión de proyectos o de seguimiento de errores.

Ejemplo



Herramientas de Ayuda a la Codificación

➤ Entorno de Desarrollo Integrado (IDE)

➤ Depuradores

➤ Control de Versiones

➤ Integración Continua

➤ Frameworks

➤ Analizadores de Código

➤ Autogeneradores de Código

¿Qué es la “Integración Continua”?

“La integración continua es una práctica en la que miembros de un equipo de desarrollo mezclan su código frecuentemente llegando a generar varias versiones por día. Cada versión ejecutable es verificada por un sistema automático de compilación y pruebas para detectar errores lo más rápido posible.”

- Martin Fowler, 2000



Problemas que la Integración Continua soluciona

- ❑ **El código únicamente compila en la estación de trabajo de un programador.** Es posible que algún desarrollador introduzca, sin saberlo, ciertas restricciones en cuanto a la plataforma de compilación del software. Cuando otro miembro del equipo descarga el código del repositorio, el código deja de compilar en su máquina.
- ❑ **El producto funciona adecuadamente en todas las estaciones de trabajo de todos los programadores pero no funciona en el servidor de producción.** La dependencia con ciertas versiones de binarios, librerías y configuraciones puede ocasionar comportamientos anómalos del software. Es posible que el equipo de desarrollo utilice unas versiones de librerías y unas configuraciones diferentes a las que el producto usará en los servidores de producción.
- ❑ **No se ejecutan las pruebas automatizadas del código.** Es posible que una vez los programadores han escrito un buen conjunto de pruebas automatizadas del código que cubren gran parte de sus funcionalidades, al equipo de desarrollo se le olvida ejecutarlas.

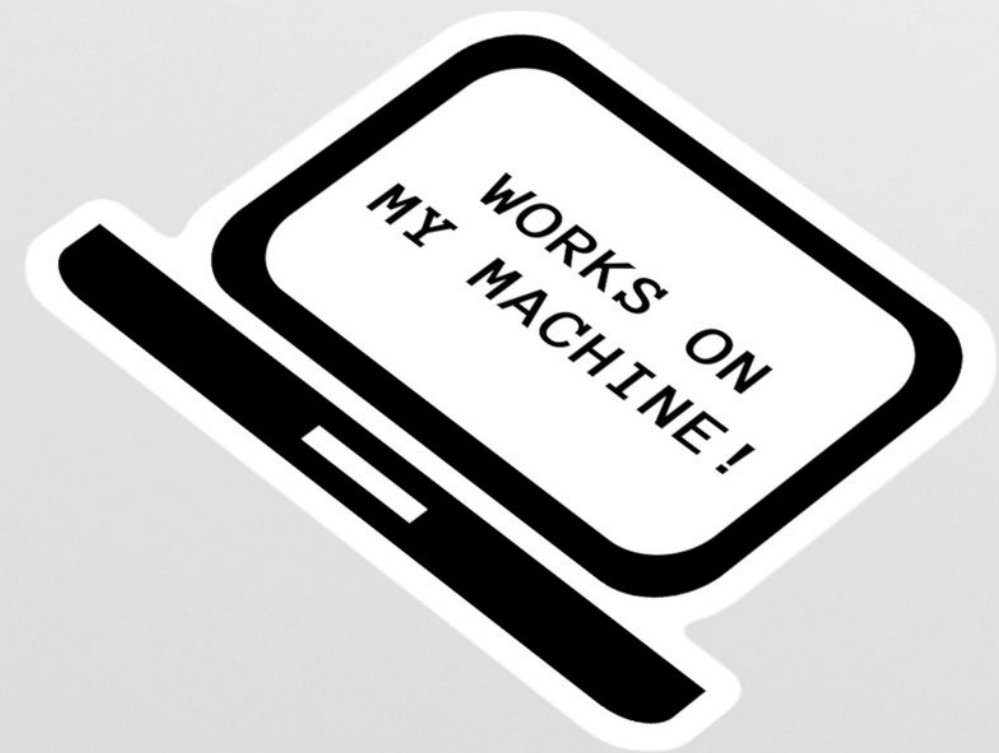
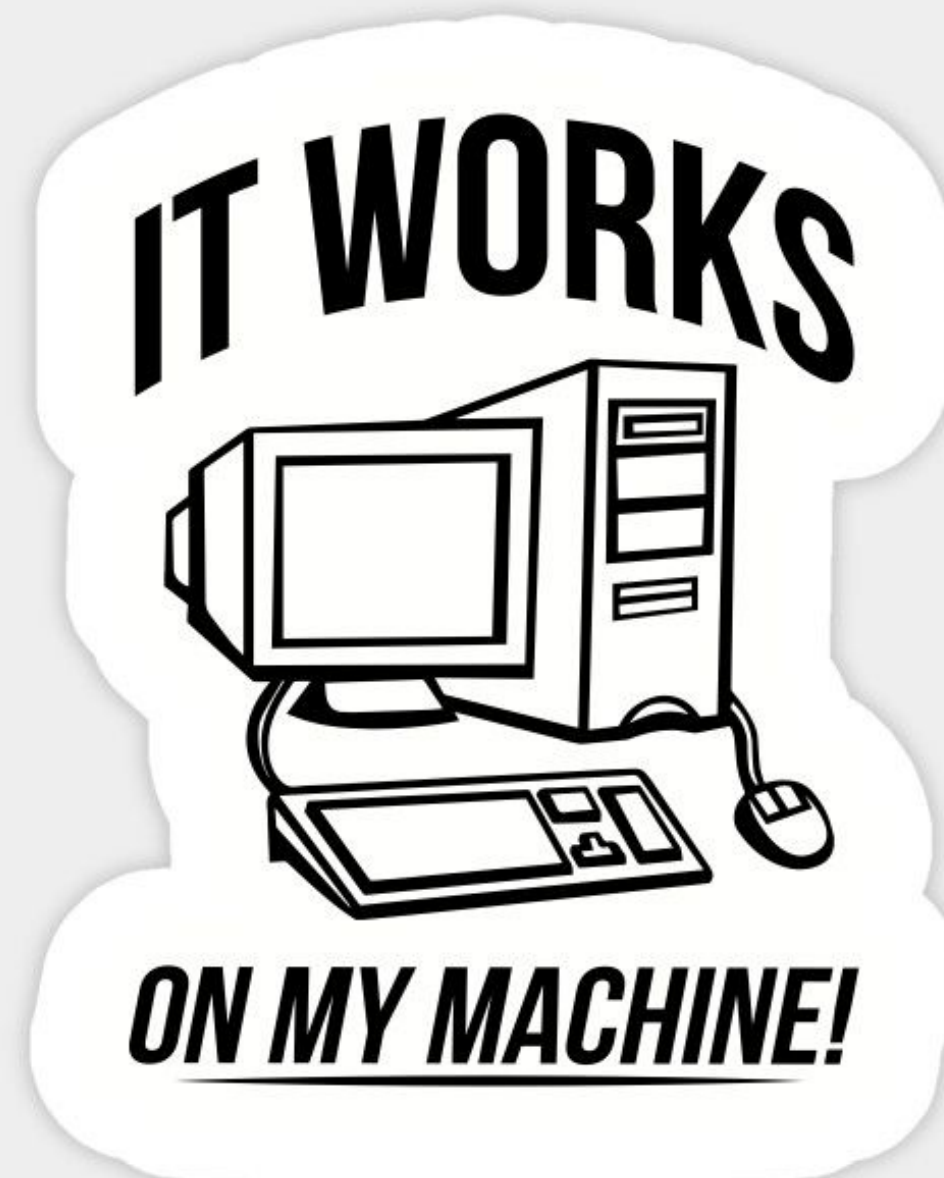
Problemas que la Integración Continua soluciona



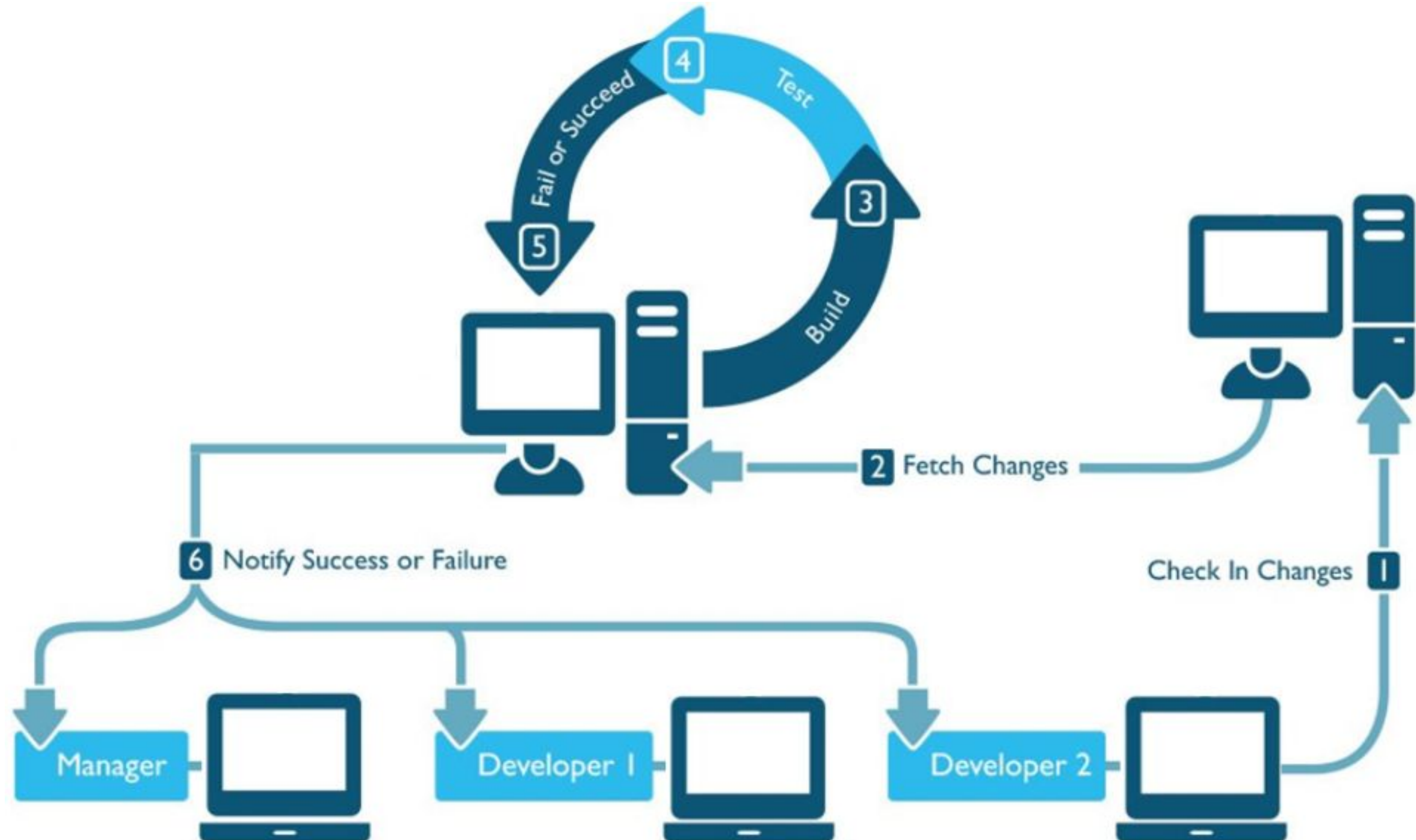
LA FRASE POR EXCELENCIA
DEL PROGRAMADOR



"¡EN MI
MÁQUINA SI
FUNCIONA!"



Cómo funciona la Integración Continua



Cómo funciona la Integración Continua

jenkins / SIM / master / #42

https://gesintegracion.admon-cfnavarra.es/ci/blue/organizations/jenkins/SIM/detail/master/42/pipeline

✓ SIM < 1.0.0.42

Pipeline Modificación Pruebas Artefacto

Rama: master 41s Modificado por Pérez Rodríguez, Gorka (AVALON, DGTD Servicio de Tecnologías de Salud)

Commit: d29eae6 in a few seconds Replayed #41

Go to classic

Start Workspace Checkout Info Change AssemblyVersion Build Tests Archive Deploy Version A Desplegar Calculo del Despliegue Nodos Guardar Despliegue End

Frontal Nodo 1

Frontal Nodo 2

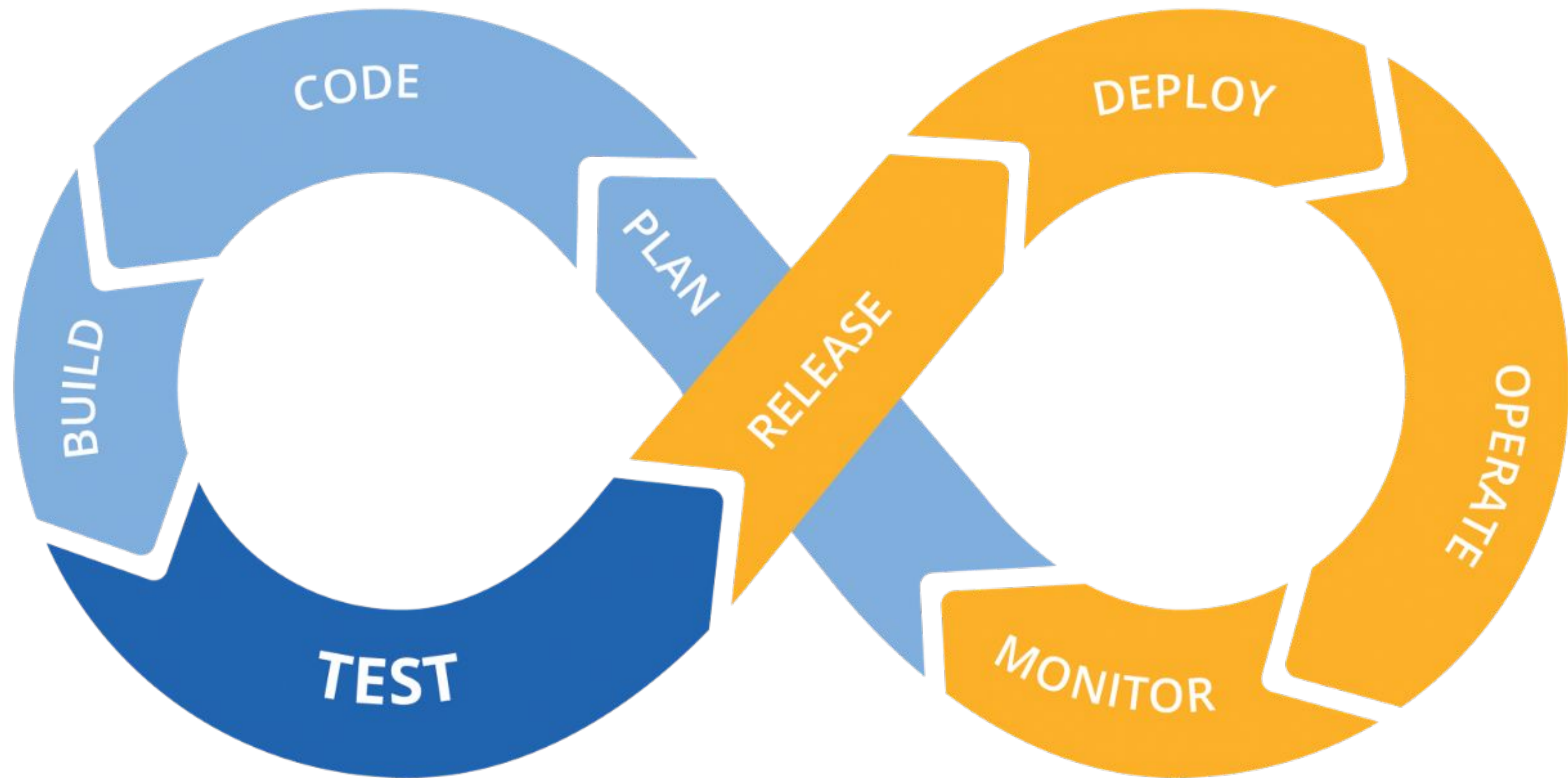
Guardar Despliegue - 3s

Restart Guardar Despliegue

- ✓ > C:\GAD\DeployMapLibraryManager\JenkinsComponentLibraryManager.Con.exe — Verify if file exists in workspace <1s
- ✓ > C:\GAD\DeployMapLibraryManager\JenkinsComponentLibraryManager.Con.exe SaveDeploy [{"fullPath":"D:\GN\SSIAS\SIM\master\GN.Sanidad.SIM.Mvc","branchName":"master","componentName":"GN.Sanidad.SIM.Mvc","server":"CENTPDV01-D..."}] — Windows Batch Script 2s
- ✓ > rmdir D:\WTOBYTJT /s /q — Windows Batch Script <1s
- ✓ > rmdir D:\WTOBYTJT@tmp /s /q — Windows Batch Script <1s

Cómo funciona la Integración Continua

Entrega Continua (CD)



Integración Continua (CI)

Herramientas de Ayuda a la Codificación

- Entorno de Desarrollo Integrado (IDE)
- Depuradores
- Control de Versiones
- Integración Continua
- **Frameworks**
- Analizadores de Código
- Autogeneradores de Código

¿Qué es un framework?

De manera general, la wikipedia define framework como “un conjunto estandarizado de conceptos, prácticas y criterios para enfocar un tipo de problemática particular que sirve como referencia, para enfrentar y resolver nuevos problemas de índole similar”.

Las principales ventajas que ofrecen los frameworks son:

- ❑ Simplifica el desarrollo. El framework ofrece “*precocinada*” la estructura de la aplicación (patrón de presentación, arquitectura de capas, etc.). El desarrollador o diseñador se limita a seguir la “*plantilla*” ahorrando líneas de código.
- ❑ Facilita el mantenimiento y la extensión del código. El framework guía al programador a implementar buenas prácticas de diseño y codificación. Además mejora la colaboración entre desarrolladores, el código fuente sigue una misma estructura base independientemente del desarrollador que lo escribió.
- ❑ Reduce errores de desarrollo (bugs), al reducir el número de líneas y facilitar la reutilización de código.
- ❑ Ayuda a implementar productos software más seguros ya que ofrece soluciones estandarizadas a vulnerabilidades recurrentes. Por ejemplo, casi todos los frameworks ofrecen filtros anti-XSS por defecto.

Frameworks

Existe una enorme cantidad de frameworks disponibles. Existen frameworks para muchos lenguajes. Algunos son mantenidos por grupos de desarrolladores de forma totalmente altruista, lo hacen por el placer de compartir con la comunidad. Otros son mantenidos por grandes multinacionales como Google, Facebook o Microsoft. Algunos ejemplos de frameworks pueden ser:

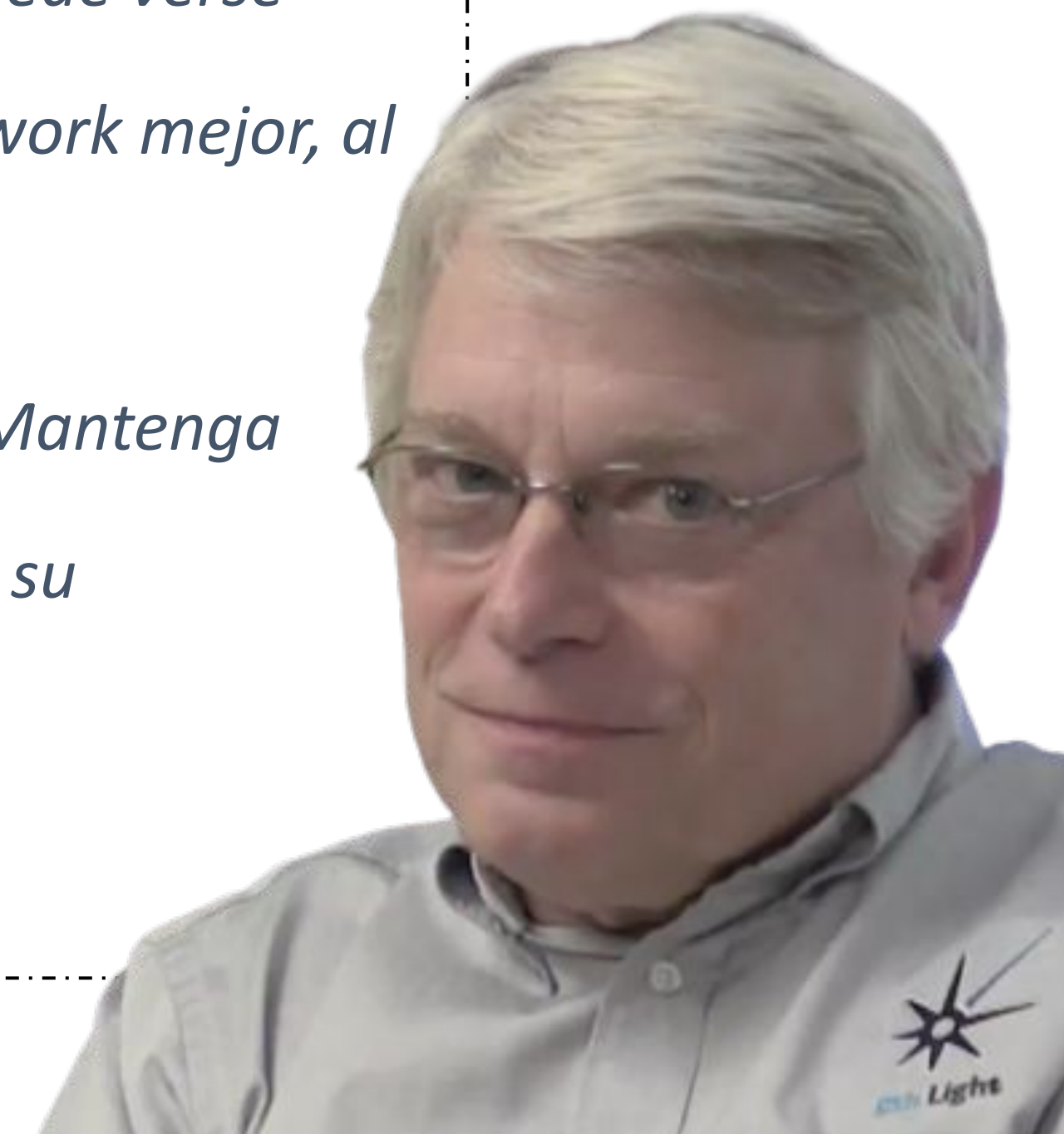
- ❑ **Swing.** Es una biblioteca gráfica para Java. Incluye widgets para interfaz gráfica de usuario tales como cajas de texto, botones, listas desplegables y tablas.
- ❑ **ASP.NET MVC.** Es un framework de aplicaciones web que implementa el patrón modelo-vista-controlador (MVC) desarrollado por Microsoft.
- ❑ **Angular.** Es un framework para aplicaciones web desarrollado en TypeScript, de código abierto, mantenido por Google, que se utiliza para crear y mantener aplicaciones web de una sola página (SPA, single page application). Su objetivo es ayudar a implementar el patrón MVC en aplicaciones que corren en el navegador.

Frameworks

“La relación entre usted y el autor del framework es muy asimétrica. Debe comprometerse enormemente con el framework, pero el autor del framework no se compromete en absoluto con usted. [...] el autor del framework le insta a acoplar su aplicación al framework, tan estrechamente como sea posible. Para el autor del framework, el acoplamiento a su framework no supone ningún riesgo. [...] El autor le está pidiendo que se case con el framework; que realice un enorme y duradero compromiso con el framework. ¿Cuáles son los riesgos? [...] El framework puede evolucionar en una dirección que no le resulte de utilidad. Puede verse atrapado actualizando a nuevas versiones que no le ayudan. Puede aparecer un nuevo framework mejor, al que le gustaría cambiar.

¡No se case con el framework! Puede utilizar el framework; simplemente no se acople con él. Mantenga cierta distancia. Trate el framework como un detalle que pertenece a los círculos exteriores de su arquitectura.”

- Robert C. Martin, Arquitectura Limpia



Herramientas de Ayuda a la Codificación

- Entorno de Desarrollo Integrado (IDE)
- Depuradores
- Control de Versiones
- Integración Continua
- Frameworks
- Analizadores de Código
- Autogeneradores de Código

¿Qué son los analizadores de código?

Los analizadores de código son herramientas, aplicaciones software, capaces de medir la calidad del código automáticamente.

El análisis de código puede ser estático o dinámico:

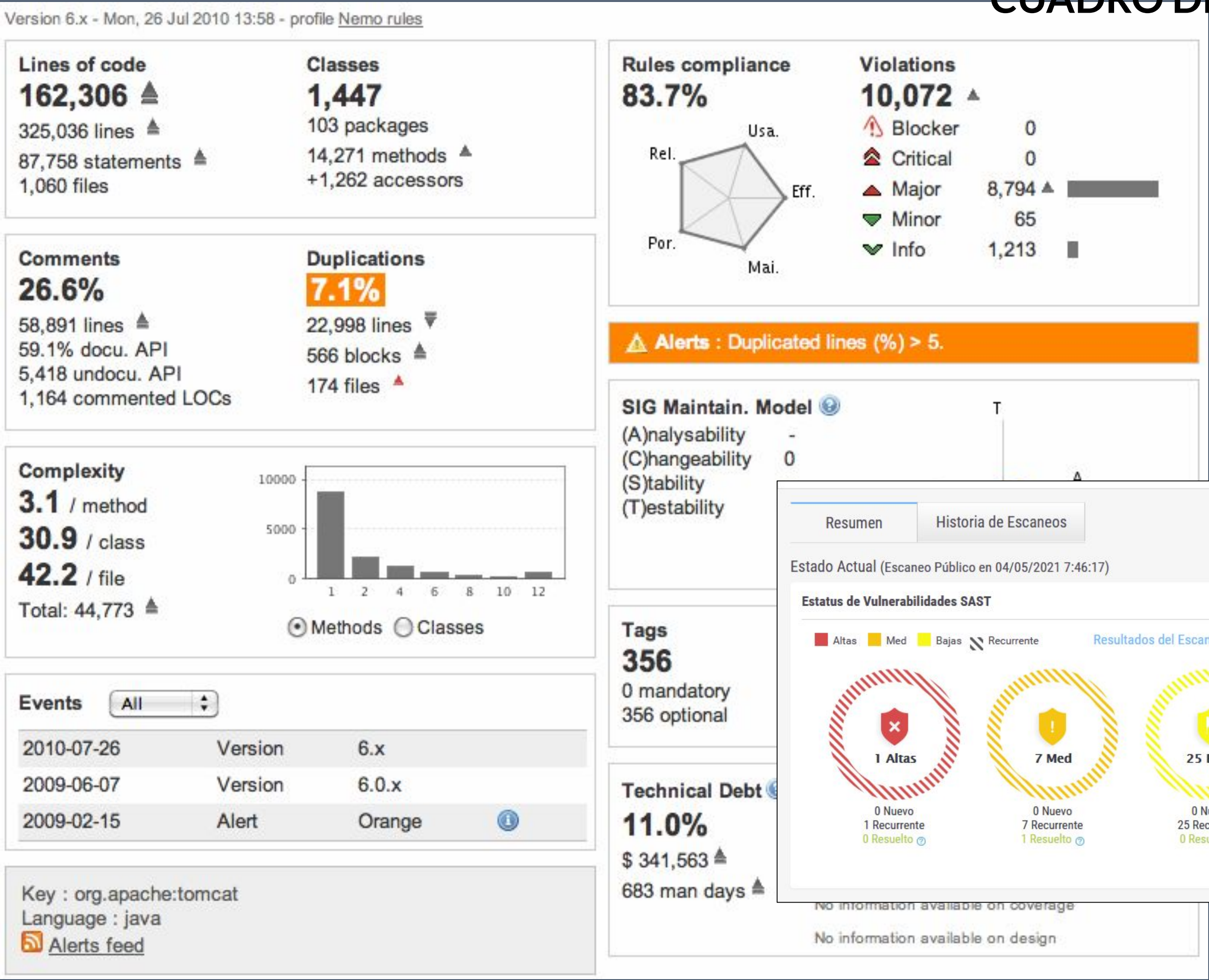
- ❑ El analizador estático lee el código sin necesidad de ejecutarlo. Como no necesita de esa ejecución, el análisis estático permite detectar errores en una fase muy temprana de la codificación.
- ❑ El analizador dinámico observa el código mientras se está ejecutando. Es más lento y necesita un proceso completo de testeo. A cambio, permite detectar errores que quedan ocultos en un análisis estático.

Métricas evaluadas por los Analizadores Estáticos

- ❑ **Índice de mantenimiento.** Calcula un valor de índice entre 0 y 100 que representa la facilidad relativa de mantenimiento del código.
- ❑ **Complejidad ciclomática.** Mide la complejidad estructural del código. Se crea calculando el número de rutas de acceso de código diferentes en el flujo del programa.
- ❑ **Profundidad de herencia.** Indica el número de clases diferentes que heredan de otra, hasta la clase base. Cuanto mayor sea este número, más profunda será la herencia y mayor será la posibilidad de que las modificaciones de la clase base produzcan cambios importantes.
- ❑ **Acoplamiento de clases.** Mide la dependencia entre clases a través de parámetros, variables locales, tipos de valor devueltos, llamadas a métodos, implementaciones de interfaces, etc. Un buen diseño de software dicta que los tipos y métodos deben tener una cohesión alta y un acoplamiento bajo.
- ❑ **Líneas de código fuente.** Indica el número exacto de líneas de código fuente que definen la clase. Una gran cantidad de líneas es síntoma de baja cohesión o de un reparto equivocado de responsabilidad (la clase ofrece una funcionalidad excesiva).

Métricas evaluadas por los Analizadores Estáticos

CUADRO DE MANDOS DE SONARQUEBE



CUADRO DE MANDOS DE CHECKMARX



Métricas evaluadas por los Analizadores Estáticos

SonarQube dispone de plugins y extensiones que se integran en el IDE facilitando al desarrollador la inspección de la calidad del código.



SonarQube detecta como riesgo de denegación de servicio cualquier expresión regular “larga”. Algunas expresiones regulares particularmente largas ofrecen un rendimiento malo, la evaluación es “pesada”. Esto puede dar lugar a un ataque por denegación de servicio si un usuario malintencionado introduce “ingeniosas” cadenas de texto que acentúan los problemas de rendimiento de la expresión regular

Comportamiento detectado por los Analizadores Dinámicos

- ❑ Uso de memoria no inicializada.
- ❑ Lectura/escritura de memoria que ha sido previamente liberada.
- ❑ Lectura/escritura fuera de los límites de bloques de memoria dinámica.
- ❑ Fugas de memoria. Un bloque de memoria reservada que ha dejado de usarse no es liberado para su reutilización

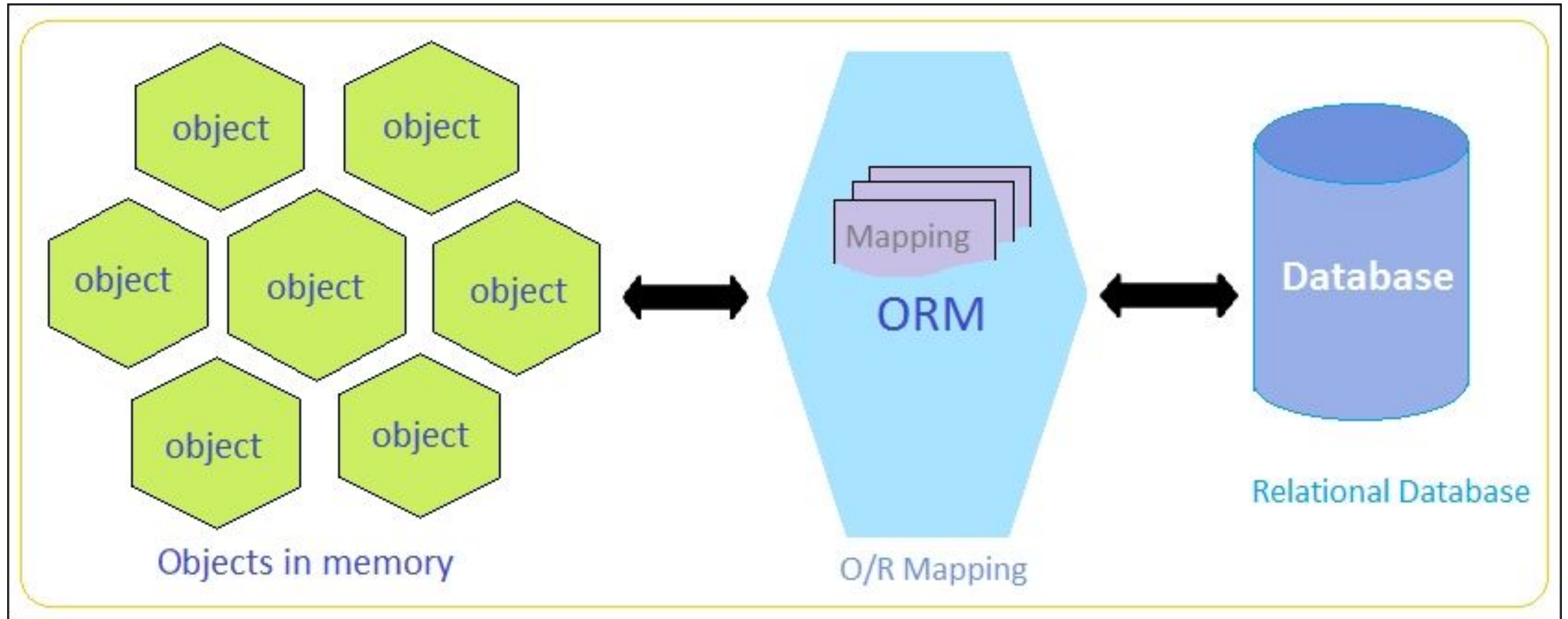
Herramientas de Ayuda a la Codificación

- Entorno de Desarrollo Integrado (IDE)
- Depuradores
- Control de Versiones
- Integración Continua
- Frameworks
- Analizadores de Código
- Autogeneradores de Código

¿Qué es la autogeneración de código?

- ❑ La generación de código de forma automática permite, en algunos casos, incrementar la productividad al evitar tener que escribir el código de la especificación y mantener un estilo del código uniforme y consistente.
- ❑ Como principal desventaja está la dificultad en mantener código auto-generado mezclado con código propio.
- ❑ Los generadores de código requieren como input, especificaciones con un grado de detalle muy alto.

Ejemplo: ORMs



Más ejemplos

- ❑ Herramientas CASE que a partir de diagramas UML generan el código y permiten mantener sincronizado en ambos sentidos.
- ❑ Los IDEs modernos son capaces de generar código “de fontanería” (boilerplate) como son los getters y setters para las propiedades públicas.
- ❑ Herramientas integradas en el IDE para dibujar interfaces de usuario que generan el esqueleto del código que se corresponde con los componentes dibujados, como por ejemplo Swing en Netbeans.

